ESTUDO DIRIGIDO · M3 · AULA 1 DE 3 · M3 · WORKFLOWS, AGENTES, TOOL USE E MCPS

Workflow ou agente: quem decide o próximo passo

Ao fim deste estudo você sabe dizer, para um problema concreto, se ele pede um workflow ou um agente, monta os dois no caso Balcão com o Claude Code e escreve a política de parada de um agente.

Aula de Qua, 11/11/2026 | Tempo sugerido: cerca de 3 horas, em uma ou duas sentadas | Sistemas com IA para Devs · CESAR School

Como usar este estudo

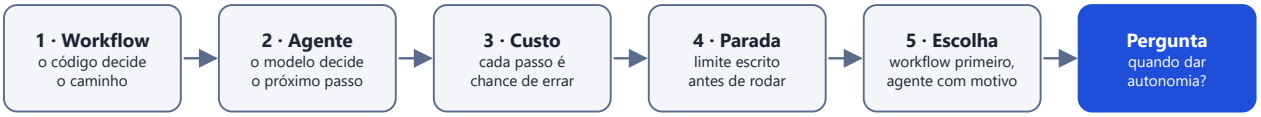
Primeiro leia os conceitos na ordem: cada um sai do anterior, como na aula. Depois faça as práticas, que repetem no seu computador o que o professor demonstrou, com o Claude Code e o caso Balcão. No fim, confira a autoavaliação e as respostas das perguntas.

ANTES DE COMEÇAR

- [ ] Claude Code instalado e com login feito. Teste no terminal: `claude -p "responda apenas: ok"` precisa responder `ok`.
- [ ] Python 3.11 ou mais novo (`python --version`).
- [ ] A pasta `caso-balcao` do repositório da turma, github.com/victorhwfreire/sistemas_ia_devs_b2c_20262, no seu computador. Os comandos abaixo partem de `caso-balcao/ferramentas`.
- [ ] No Windows PowerShell, rode antes `[Console]::OutputEncoding = [Text.Encoding]::UTF8` para os acentos saírem certos.

O mapa da aula

Pergunta da aula: **quando vale dar autonomia ao modelo?**



Mapa da aula: cinco conceitos em cadeia. No workflow, o código decide o caminho; no agente, o modelo decide o próximo passo; cada passo a mais é uma chance a mais de errar; a política de parada é um limite escrito antes de rodar; e a escolha é workflow primeiro, agente com motivo. Termina na pergunta destacada: quando vale dar autonomia ao modelo?

Conceito 1 de 5

No workflow, o código decide o caminho

> **Workflow é código que chama o modelo em passos fixos: a ordem, os desvios e o fim estão escritos antes de rodar.**

Um sistema com LLM raramente é uma chamada só. Na triagem do Balcão, uma chamada classifica o ticket, uma regra decide o time e outra chamada escreve a resposta. Quando a sequência é escrita no código, temos um workflow. O modelo trabalha dentro de cada passo; a passagem de um passo para o outro é do código.

O RAG do M2 é um workflow: buscar trechos, montar o contexto, gerar a resposta, sempre nessa ordem. A Anthropic descreve cinco padrões que cobrem a maior parte dos workflows: encadeamento (a saída de uma chamada vira entrada da próxima), roteamento (uma classificação escolhe o caminho), paralelização (várias chamadas ao mesmo tempo, depois junta), orquestrador e trabalhadores (uma chamada divide a tarefa e outras executam) e avaliador e otimizador (uma chamada gera, outra critica e pede revisão).

A vantagem do workflow é que ele é previsível. Dá para desenhar o fluxo antes de rodar, testar cada passo isolado, prever custo multiplicando chamadas por tickets e explicar depois por que algo aconteceu. A limitação é a mesma: ele só faz o que está escrito. Se o ticket pede uma investigação que não estava nos passos, ninguém investiga.



Workflow de triagem em cinco etapas: o ticket entra como texto; o modelo classifica categoria e prioridade; o código roteia por uma regra fixa por categoria; o modelo redige a resposta ao cliente; e o ticket chega à fila do time, sempre em três passos.

NO CASO BALCÃO

O `workflow_triagem.py` do caso faz duas chamadas ao modelo por ticket (classificar e responder) e uma decisão em código (o dicionário ROTA, que leva cada categoria a um time). Para doze tickets, são 24 chamadas, sempre.

CONFUSÃO FREQUENTE

Achar que, por usar um LLM, o sistema é um agente. Se o caminho está no código, é workflow, mesmo que cada passo seja uma chamada sofisticada.

PARA CONFERIR · RESPOSTA NO FIM

No workflow da triagem, qual passo não usa o modelo, e por que isso é bom?

Conceito 2 de 5

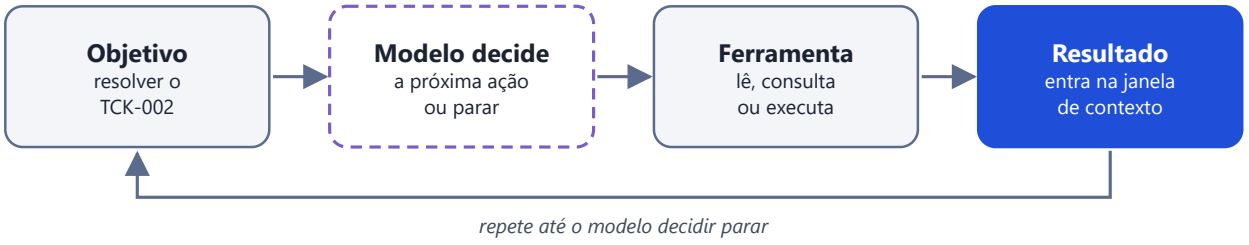
No agente, o modelo escolhe o próximo passo

> **Agente é um loop: o modelo olha o objetivo e o que já aconteceu, escolhe uma ação, recebe o resultado e decide de novo, até decidir parar.**

No agente, você entrega ao modelo um objetivo e um conjunto de ferramentas, por exemplo ler arquivos ou consultar uma base. O modelo decide qual ação tomar. Seu código executa essa ação e devolve o resultado, que entra na janela de contexto. O modelo olha a janela, agora maior, e decide de novo. Em pseudocódigo: enquanto o modelo não disser que terminou, chame o modelo, execute o que ele pediu e devolva o resultado.

Esse padrão foi descrito em 2022 no artigo ReAct, que alterna raciocinar e agir. O Claude Code que você usa é um agente desse tipo: um modelo num loop com ferramentas para ler, editar e rodar comandos. Quando você pede uma mudança no código, ele decide sozinho quais arquivos abrir, o que testar e quando parar.

O modelo do agente é o mesmo do workflow. O que muda é quem decide a próxima ação. E decidir a próxima ação inclui decidir quantas ações tomar: é daí que vêm a flexibilidade e o custo do agente.



Loop do agente: o objetivo, resolver o TCK-002, vai ao modelo, que decide a próxima ação ou parar; a ferramenta lê, consulta ou executa; o resultado entra na janela de contexto; e o ciclo se repete até o modelo decidir parar.

NO CASO BALCÃO

Com o objetivo "resolva o TCK-002" e permissão só de leitura na pasta `dados`, o agente das execuções registradas para o guia listou a pasta, leu a política, os clientes e as cobranças de uma vez, achou dois lançamentos de R$ 189,90 em 05/10 (COB-5002 e COB-5003) e recomendou estornar o COB-5003. Sem o pedido mandar, também apontou a mesma duplicidade no Bazar Estrela.

CONFUSÃO FREQUENTE

Chamar de agente qualquer chatbot. Um chat que só responde, sem ferramentas e sem loop de ação, é uma chamada ao modelo, não um agente.

PARA CONFERIR · RESPOSTA NO FIM

Por que o resultado de cada ferramenta precisa voltar para a janela de contexto?

Conceito 3 de 5

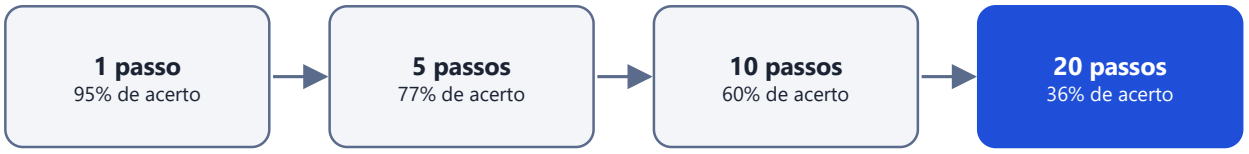
Cada passo a mais é uma chance a mais de errar

> **Autonomia custa tempo, dinheiro e risco: o número de passos vira variável e o erro de cada passo se acumula.**

Cada volta do agente é mais uma chamada ao modelo. Ela custa tempo, porque espera a resposta; custa dinheiro, porque reenvia a janela inteira, que cresce a cada volta; e custa risco, porque cada decisão pode estar errada.

O risco se acumula. Se cada passo acerta 95% das vezes e os passos fossem independentes, uma execução de 5 passos acertaria $0{,}95^5 \approx 77\%$; de 10 passos, $0{,}95^{10} \approx 60\%$; de 20 passos, $0{,}95^{20} \approx 36\%$. Na prática os passos não são independentes, e o agente às vezes corrige um erro anterior, mas a direção vale: cadeias longas erram mais.

O segundo efeito é a variação. O mesmo ticket, rodado duas vezes, pode levar três passos numa execução e seis na outra. Custo e tempo passam a ter uma faixa, não um valor. No workflow, o número de chamadas por ticket é fixo.



Erro composto, ilustrativo: com 95% de acerto em cada passo independente, uma execução de 1 passo acerta 95%; de 5 passos, 77%; de 10 passos, 60%; de 20 passos, 36%.

NO CASO BALCÃO

Nas duas execuções registradas do TCK-002, o agente seguiu o mesmo caminho, mas escreveu respostas diferentes ao cliente: uma disse que o estorno "está aprovado", a outra, "vamos estornar". Com mais dados e mais ferramentas, o caminho também varia. Compare os seus resumos com `ler_execucao.py --json`.

CONFUSÃO FREQUENTE

Medir um agente com uma execução só. Uma execução boa não diz quanto ele custa nem quantas vezes acerta; isso é tema do M4.

PARA CONFERIR · RESPOSTA NO FIM

Com 90% de acerto por passo, quanto acerta uma execução de 10 passos independentes?

Conceito 4 de 5

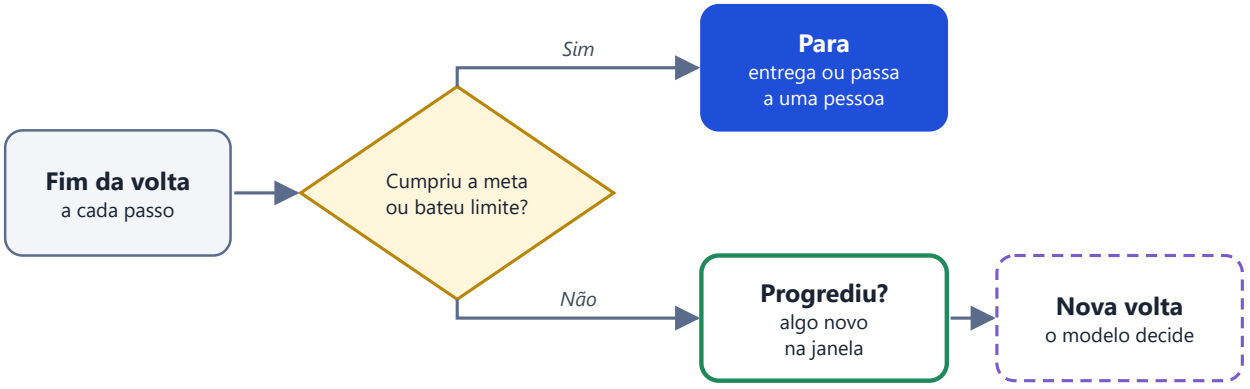
Todo agente precisa de uma política de parada

> **Política de parada é a regra, escrita antes de rodar, de quando o loop termina: objetivo cumprido, limite atingido ou passagem para uma pessoa.**

Se cada volta custa e arrisca, alguém precisa decidir quando o loop acaba, e não pode ser só o modelo. A política de parada tem cinco campos: objetivo verificável (como sei que terminou), teto de passos, teto de custo, teto de tempo e condição de escalada para uma pessoa.

A condição de escalada cobre dois casos. Falta de progresso: duas voltas sem informação nova na janela é sinal de que o agente está andando em círculo. Ação sensível: um estorno, uma exclusão, um e-mail para o cliente. Nesses casos, o agente para e passa o caso adiante com o que já descobriu.

No Claude Code em modo não interativo (`claude -p`), `--max-budget-usd` limita o gasto estimado e `--max-turns` limita as voltas. O corte acontece depois da chamada que cruzou o limite, então o gasto final pode passar um pouco do teto. A política não deixa o agente mais inteligente; ela garante que o erro tem tamanho máximo.



Decisão ao fim de cada volta: cumpriu a meta ou bateu um limite? Se sim, para: entrega a resposta ou passa a uma pessoa. Se não, verifica se progrediu, com algo novo na janela; com progresso, uma nova volta em que o modelo decide.

NO CASO BALCÃO

Política do agente de cobrança do Balcão: objetivo = recomendação com o id do lançamento conferido; até 8 voltas; até US$ 0,50; até 2 minutos; escala para o financeiro se o estorno passar de R$ 200 ou se duas voltas seguidas não trouxerem dado novo.

CONFUSÃO FREQUENTE

Confiar que o modelo vai parar quando terminar. Em geral ele para, mas a política existe para os casos em que não para.

PARA CONFERIR · RESPOSTA NO FIM

Qual campo da política impede que um agente fique relendo o mesmo arquivo por dez voltas?

Conceito 5 de 5

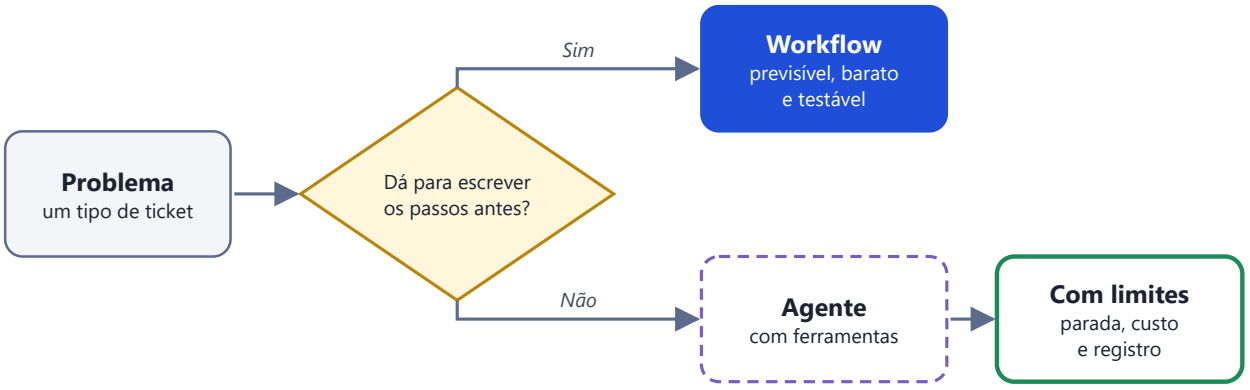
Comece pelo workflow; suba para agente com motivo

> **Se dá para escrever os passos antes, é workflow. Se o caminho depende do que se descobre no meio, é agente, com limites.**

A pergunta que decide é: dá para escrever os passos antes de rodar? Para uma dúvida de produto, dá: classificar, responder com o artigo certo, fim. Para uma cobrança contestada, o caminho depende do que aparece: é duplicidade, tarifa sem contrato, antecipação contratada? Escrever todos esses desvios antes é caro e frágil; o agente se paga.

Os dois se combinam. Um desenho comum é o workflow que roteia e chama um agente só na categoria que precisa investigar. Assim o custo e a variação do agente ficam restritos aos tickets que ganham com eles.

A recomendação da Anthropic, no texto de 2024, é começar pela solução mais simples que resolve e só acrescentar autonomia quando houver ganho medido. Muita equipe faz o contrário: começa pelo agente porque impressiona na demonstração e descobre o custo em produção.



Decisão de síntese: para um problema, como um tipo de ticket, dá para escrever os passos antes? Se sim, workflow: previsível, barato e testável. Se não, agente com ferramentas, sempre com limites de parada, custo e registro.

NO CASO BALCÃO

No Balcão: `duvida_produto` e `falha_transacao` com passos conhecidos vão por workflow; `cobranca_indevida`, que exige conferir lançamentos e contrato, vai para o agente com teto de US$ 0,50 e escalada acima de R$ 200.

CONFUSÃO FREQUENTE

Tratar agente como upgrade natural de workflow. É outra decisão de projeto, com outro custo.

PARA CONFERIR · RESPOSTA NO FIM

Um sistema que sempre extrai os mesmos cinco campos de uma nota fiscal pede workflow ou agente?

Glossário da aula

TERMO	O QUE É
Workflow	código que chama o modelo em passos fixos.
Agente	loop em que o modelo escolhe a próxima ação até decidir parar.
Ferramenta (tool)	função que o modelo pode pedir para executar: ler, consultar, gravar.
Janela de contexto	tudo o que o modelo recebe numa chamada; no agente, cresce a cada volta.
Encadeamento	padrão em que a saída de uma chamada vira entrada da próxima.
Roteamento	padrão em que uma classificação escolhe o caminho seguinte.
Saída estruturada	resposta forçada a seguir um esquema JSON; no Claude Code, `--json-schema`.
Erro composto	queda do acerto total quando vários passos com erro se encadeiam.
Política de parada	regra escrita de quando o loop termina.
Escalada	passagem do caso para uma pessoa, com o que o agente já descobriu.
`claude -p`	Claude Code em modo não interativo: recebe o pedido, executa e devolve o resultado.
`--output-format json` e `stream-json`	saída com resultado, tempo e custo; o segundo traz cada passo da execução.

Prática 1 · Monte o workflow de triagem

50 min

Objetivo. Construir, com o Claude Code, um workflow de três passos para os tickets do Balcão e medir chamadas, tempo e custo.

1. Abra um terminal na pasta `caso-balcao` e faça uma chamada isolada ao modelo, pedindo JSON.

```
claude -p "Classifique este ticket em uma palavra: maquininha não passa débito desde as 9h." --output-format json
```

2. Leia o JSON devolvido e localize `result` (a resposta), `duration_ms` (o tempo) e `total_cost_usd` (o custo estimado). Em assinatura, o custo é estimativa, não cobrança.

3. Abra o Claude Code na pasta (`claude`) e peça o script, com este texto:

TEXTO PARA O CLAUDE CODE

Crie meu_workflow.py em ferramentas/. Para cada ticket de tickets.json, faça três passos: (1) chame `claude -p` com --output-format json e --json-schema para obter categoria (uma das categorias de tickets.json) e prioridade (P1, P2 ou P3), usando a política de dados/politica-atendimento.md como instrução; (2) escolha o time com um dicionário Python, sem modelo; (3) chame `claude -p` de novo para redigir uma resposta de três frases ao cliente. Mande o pedido pela entrada padrão. Imprima uma tabela com ticket, categoria, prioridade, time, tempo e custo, e compare categoria e prioridade com o gabarito.

4. Antes de aceitar, leia o código que ele propôs e confira: o roteamento está em código? O pedido vai pela entrada padrão? Depois rode em três tickets.

```
python ferramentas/meu_workflow.py TCK-001 TCK-002 TCK-011
```

5. Compare com a versão de referência do caso, que faz o mesmo.

```
python ferramentas/workflow_triagem.py TCK-001 TCK-002 TCK-011
```

O QUE OBSERVAR

- O número de chamadas por ticket é sempre o mesmo.
- Tempo e custo ficam numa faixa estreita de um ticket para o outro.
- O TCK-011 (a farmácia) é classificado, mas ninguém investiga se foi tarifa ou erro.

ESTÁ PRONTO QUANDO

O seu script roda nos doze tickets e imprime a tabela com acerto contra o gabarito.

PARA IR ALÉM

Troque o modelo com `--model haiku` nas chamadas e compare tempo, custo e acerto.

Prática 2 · Rode o mesmo ticket como agente e compare 45 min

Objetivo. Ver o loop do agente num registro de execução e medir a variação entre execuções do mesmo pedido.

1. Rode o agente no TCK-002, com leitura liberada só na pasta `dados` e teto de US$ 0,50.

```
cd ferramentas
python rodar_agente.py --ticket TCK-002 --modo arquivos --saida ag1.jsonl
```

2. Leia o resumo: quais arquivos ele leu, em que ordem, quantas voltas, quanto tempo e quanto custou.

3. Rode de novo, duas vezes, gravando em arquivos diferentes.

```
python rodar_agente.py --ticket TCK-002 --modo arquivos --saida ag2.jsonl
python rodar_agente.py --ticket TCK-002 --modo arquivos --saida ag3.jsonl
```

4. Compare as três execuções em uma linha cada.

```
python ler_execucao.py ag1.jsonl --json
python ler_execucao.py ag2.jsonl --json
python ler_execucao.py ag3.jsonl --json
```

5. Agora rode o TCK-004, uma dúvida simples, como agente e como workflow, e compare custo e tempo.

```
python rodar_agente.py --ticket TCK-004 --modo arquivos --saida ag-duvida.jsonl
python workflow_triagem.py TCK-004
```

6. Abra `ag1.jsonl` num editor e encontre um bloco `tool_use` e o `tool_result` correspondente: é o formato do loop que a aula 2 vai explicar.

O QUE OBSERVAR

- O agente decide o que ler; o pedido não diz quais arquivos abrir.
- Entre as três execuções do TCK-002, compare voltas, tempo, custo e o texto da resposta: o que variou e o que se repetiu?
- Para a dúvida simples, o agente custa mais e não entrega mais que o workflow.

ESTÁ PRONTO QUANDO

Você preencheu uma tabela com voltas, arquivos lidos, tempo e custo das três execuções e escreveu em uma frase quando o agente compensou.

PARA IR ALÉM

Faça o agente ler um arquivo que não está liberado (peça no `--pedido` para editar a política) e encontre a negação no registro.

Prática 3 · Escreva a política de parada de um agente seu

35 min

Objetivo. Transformar a política de parada em texto concreto para um agente do seu contexto de trabalho e testar um teto.

1. Escolha uma tarefa do seu trabalho que você daria a um agente. Escreva em um parágrafo o objetivo e as ferramentas que ele teria.
2. Preencha os cinco campos: objetivo verificável; teto de passos; teto de custo; teto de tempo; condição de escalada (falta de progresso e ação sensível).
3. Peça ao Claude uma crítica da sua política, com este texto (troque os colchetes):

   TEXTO PARA O CLAUDE CODE

   Sou dev e vou colocar um agente em produção. Tarefa: [descreva]. Ferramentas: [liste]. Política de parada: [cole os cinco campos]. Aponte, em no máximo cinco itens, situações em que essa política deixaria o agente rodar demais ou parar cedo demais, e sugira um ajuste para cada uma.

4. Teste um teto de verdade no caso Balcão: rode o TCK-011 com orçamento muito baixo e veja como a execução termina.

   ```
   python rodar_agente.py --ticket TCK-011 --modo arquivos --orcamento 0.01 --saida teto.jsonl
   ```

5. No resumo, confira o campo de fim (`subtype`) e o custo final. Ele passou do teto? Explique por quê em uma frase.

O QUE OBSERVAR

- O corte acontece depois da chamada que cruza o limite.
- Uma política sem condição de escalada só sabe parar, não sabe para quem passar o caso.

ESTÁ PRONTO QUANDO

Você tem a política de parada em cinco campos, revisada com a crítica do Claude, e entende como o teto de orçamento termina uma execução.

PARA IR ALÉM

Desenhe o híbrido do Balcão: workflow para todas as categorias e agente só para `cobranca_indevida`. Quanto custaria o lote de doze tickets nesse desenho?

Autoavaliação

Marque o que você já consegue fazer sem consultar o material. O que ficar em branco indica o conceito para reler.

- [ ] Consigo dizer, olhando um sistema, quem decide o próximo passo: o código ou o modelo.
- [ ] Consigo citar três dos cinco padrões de workflow e um exemplo de cada no Balcão.
- [ ] Consigo explicar o loop do agente e por que o resultado da ferramenta volta para a janela.
- [ ] Consigo calcular o erro composto de uma cadeia de passos e dizer por que é só uma aproximação.
- [ ] Consigo escrever os cinco campos de uma política de parada.
- [ ] Consigo justificar workflow ou agente para um tipo de problema com a pergunta "dá para escrever os passos antes?".

Respostas das perguntas

1. No workflow da triagem, qual passo não usa o modelo, e por que isso é bom?
O roteamento. Uma regra fixa por categoria é determinística, gratuita e testável; usar o modelo ali só traria custo e variação.

2. Por que o resultado de cada ferramenta precisa voltar para a janela de contexto?
Porque o modelo só decide com o que está na janela. Se o resultado não volta, a próxima decisão é tomada às cegas.

3. Com 90% de acerto por passo, quanto acerta uma execução de 10 passos independentes?
$0{,}9^{10} \approx 35\%$. Um pouco mais de uma em cada três execuções.

4. Qual campo da política impede que um agente fique relendo o mesmo arquivo por dez voltas?
A condição de falta de progresso, reforçada pelo teto de passos.

5. Um sistema que sempre extrai os mesmos cinco campos de uma nota fiscal pede workflow ou agente?
Workflow. Os passos são conhecidos antes; um agente só traria variação.

Para ler depois

- Anthropic, "Building effective agents" (19/12/2024). https://www.anthropic.com/engineering/building-effective-agents — definições de workflow e agente e os cinco padrões.
- Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models" (ICLR 2023). https://arxiv.org/abs/2210.03629 — o artigo que descreveu o loop de raciocinar e agir.
- Claude Code, modo não interativo (headless). https://code.claude.com/docs/en/headless — `claude -p`, formatos de saída e saída estruturada.

- Claude Code, referência da CLI. https://code.claude.com/docs/en/cli-reference — opções como --max-budget-usd, --max-turns e --allowedTools.